Ajna ERC-4626 Vaults + Chainlink CRE: Adversarial Audit Q&A Compendium

An implementation-aware reference covering all 85 adversarial audit questions with direct answers across architecture, accounting integrity, liquidity risk, keeper operations, CRE design, and stress scenarios.

Table of Contents

A. Architecture and Capital Flow

1. What is the exact end-to-end flow for:

- deposit
- mint
- withdraw
- redeem
- rebalance
- emergency pause / recovery

Deposit/mint run through `Vault.deposit`/`Vault.mint` -> `_deposit` -> `BUFFER.addQuoteToken` and LP accounting updates; withdraw/redeem run through `Vault.withdraw`/`Vault.redeem` -> `_withdraw` -> `BUFFER.removeQuoteToken`; rebalancing uses `move`, `moveToBuffer`, and `moveFromBuffer`; emergency flow is `pause` or `recoverCollateral` followed by `returnQuoteToken`.

2. What assets are actually held where at each stage?

- user wallet
- vault
- buffer
- Ajna buckets
- any other helper contract

At runtime, assets are split between user wallets (pre-deposit), vault-controlled balances, the Buffer reserve, and Ajna bucket LP exposure. The Buffer is the immediate redemption source; Ajna bucket value is indirect and may require rebalancing to become withdrawable.

3. Which components are onchain and which are offchain?

Onchain: Vault/Buffer/Auth/library state transitions and Ajna pool interactions. Offchain: keeper policy timing, oracle/subgraph reads, CRE scheduling/orchestration, queue execution, and bridge-triggered writes.

4. Which parts are safety-critical for users to exit?

Safety-critical for exits are Buffer solvency (`BUFFER.total`), keeper liveness and correctness, Ajna unwindability under market stress, and integrity of privileged role/key custody.

5. Is this strategy fundamentally “automated but non-custodial,” or does user safety depend on an operator-like liveness assumption?

The design is non-custodial in ownership terms, but user exit safety depends on operator-like liveness because withdrawals are buffer-only and refill is operationally managed.

B. ERC-4626 Accounting and Share Integrity

6. How is `totalAssets()` computed?

- Does it reflect realizable exit value or only accounting value?
- Can it overstate value during illiquidity, auction lock, bad debt, or stale keeper operation?

`totalAssets()` sums Buffer value + removed-collateral accounting + Ajna bucket LP-derived value, then converts WAD to asset decimals. It is accounting value, not guaranteed immediate realizable exit value, and can overstate practical liquidity in lock/bad-debt/auction stress.

7. Are there rounding, preview, inflation, donation, or first-depositor edge cases?

- Check `deposit`, `mint`, `withdraw`, `redeem`, previews, conversions, fee math, and share issuance.
- Would a sophisticated attacker be able to manipulate share price or mint/redeem fairness?

Preview and fee paths are mostly coherent, with `_decimalsOffset()` correctly overridden for non-18 decimal assets. Main residual fairness risk is timing/liquidity-state asymmetry rather than a classic first-depositor inflation exploit.

8. Is there any mismatch between:

- assets counted in `totalAssets()`
- assets actually liquid/withdrawable now
- assets withdrawable only after keeper action
- assets exposed to Ajna bucket insolvency or lockups

Yes. There is a structural mismatch between assets counted in `totalAssets`, assets liquid now (Buffer), assets liquid after keeper action, and assets exposed to Ajna insolvency/lock states.

9. Could a user receive an economically unfair share price if:

- buffer is low,
- Ajna positions are stale,
- bucket values drift,
- interest accrues between accounting reads and actions,
- or rebalancing is delayed?

Yes. Low buffer, stale positions, bucket drift, interest accrual between reads/actions, and delayed rebalancing can produce economically unfair realized outcomes versus quoted accounting value.

10. Does the buffer-only withdrawal design create structural redemption unfairness between:

- early withdrawers,
- late withdrawers,
- large users,
- and users redeeming during stress?

Yes. Buffer-only withdrawal creates first-exit advantage and late-exit reverts in stress, which is a material economic/safety issue, not just UX friction.

C. Buffer Design and Exit Liquidity

11. Is the configured Buffer ratio sufficient in realistic stress scenarios?

Not inherently. A static configured buffer ratio is not stress-adaptive and may be insufficient under fast coordinated redemptions.

12. What happens if many users redeem before the next keeper run?

If redemptions exceed current buffer before refill, `maxWithdraw`/`maxRedeem` collapse to available Buffer value and later withdrawals/redeems revert.

13. Does the strategy create a bank-run dynamic where first exits are honored and later exits revert?

Yes, this structure can produce bank-run dynamics where early exits are honored and later exits are blocked pending rebalance.

14. Is that merely a UX issue, or a material economic/safety issue?

It is material economic and safety risk because users are forced into timing-dependent liquidity access during adverse conditions.

15. Can an attacker intentionally drain or pin the buffer to grief other users?

Yes. An attacker can intentionally pressure/pin the buffer via large redemption timing, liquidity-state manipulation, or keeper-window gaming.

16. Are there any edge cases where buffer accounting can drift from true economic value?

Yes. Buffer accounting can drift from practical economic value via stale state, interest timing, non-standard token behavior assumptions, and operational lag.

D. Ajna-Specific Economic and Market Structure Risk

17. How does Ajna bucket placement affect real yield vs apparent yield?

Ajna bucket placement strongly impacts realized yield and risk; apparent accounting performance can diverge from executable risk-adjusted outcomes.

18. What assumptions does the strategy make about:

- LUP
- HTP
- bucket bankruptcy
- auction states
- range validity
- out-of-range deposits

The strategy assumes LUP/HTP boundaries remain useful safety anchors, bankruptcy/auction/debt-lock checks are timely, and range validity reflects safe earning zones.

19. Are these assumptions valid under fast price moves, toxic flow, or stressed collateral?

Only partially. Under fast price moves, toxic flow, and stressed collateral, these assumptions can fail between keeper cycles.

20. Does the strategy systematically expose depositors to adverse selection from borrowers?

Yes. As passive liquidity, the vault is exposed to adverse selection by informed borrowers during stressed or one-sided market regimes.

21. Could the “optimal bucket” logic increase concentration in the wrong place at the wrong time?

Yes. Deterministic optimal-bucket targeting can concentrate liquidity in the wrong place at the wrong time.

22. Could the strategy chase yield into buckets that are economically fragile or difficult to exit?

Yes. With stale or biased price inputs, the strategy can chase apparent yield into fragile or hard-to-exit buckets.

23. Are there scenarios where liquidity appears productive but is practically trapped?

Yes. Liquidity can appear productive in accounting while practically trapped for near-term user exits.

E. Keeper Correctness and Operational Risk

24. What exact decisions does the keeper make?

The keeper decides whether to abort, drain, rebalance between buckets, top up buffer, or deploy excess buffer, based on pause/health/range/dust/bankruptcy/debt-lock checks and target calculations.

25. Which keeper configuration values are safety-critical?

Safety-critical configs include cadence, bucket offset, buffer padding, min move amount, bankruptcy and auction windows, subgraph fail behavior, oracle mode/staleness, fixed price, and `HALT_KEEPER_IF_LUP_BELOW_HTP`.

26. Which keeper inputs are trusted?

Trusted inputs are RPC state, subgraph responses, oracle outputs (CoinGecko/Chronicle/fixed), environment configuration, and the keeper signing key.

27. What happens when:

- the keeper is offline,
- the subgraph fails,
- price data is stale,
- the wrong oracle mode is selected,
- a fixed price is misconfigured,
- bad debt is present,
- auctions are live,
- or gas estimation fails?

Offline keeper stops rebalancing; subgraph failure can fail-open or fail-closed by config; stale/wrong oracle can mis-target or abort; fixed-price misconfig causes deterministic bad policy; bad debt/auctions can abort runs; gas estimation can fall back; `LUPBelowHTP` can hard-halt keeper.

28. Are there failure modes where the keeper safely halts?

Yes. The keeper has explicit safe-stop paths (pause, bad debt, out-of-range, dusty, bankruptcy/debt-lock, and optional LUPBelowHTP halt).

29. Are there failure modes where the keeper continues but makes economically bad moves?

Yes. It can continue in economically poor modes when inputs are valid-looking but wrong (e.g., fixed misprice, accepted stale data, fail-open health dependency).

30. Could keeper delays alone cause material user harm?

Yes. Keeper delays alone can cause meaningful user harm through redemption liveness failure and adverse realized timing.

31. Does a 12-hour cadence create meaningful latency risk during volatility?

Yes. A 12-hour cadence introduces substantial latency risk during volatile conditions.

32. Does the keeper’s target-bucket logic introduce path dependency or deterministic predictability that can be exploited?

Yes. Deterministic target logic plus known cadence creates exploitable path dependency.

33. Could someone manipulate conditions around keeper timing to induce bad rebalances?

Yes. Conditions can be manipulated around run timing to induce poor rebalances or aborts.

34. Are there race conditions between state read, decision, and transaction inclusion?

Yes. Read-decide-write race windows exist between observed state and transaction inclusion.

F. Oracle / Price Source / Data Dependency Risk

35. Even though Ajna is described as oracle-less, where does this implementation reintroduce oracle dependence?

Oracle dependence is reintroduced in offchain keeper bucket targeting (`getPrice` path), even if Ajna core is oracle-less.

36. Compare the risk of:

- CoinGecko/API path
- Chronicle/onchain oracle path
- fixed price path
- CRE + Chainlink feed path

CoinGecko path has API centralization/outage risk; Chronicle path has stronger integrity but staleness/coverage caveats; fixed-price path has highest operator misconfiguration risk; CRE+Chainlink feed path is strongest if integrated as deterministic policy input.

37. Which path is safest?

Safest target architecture is deterministic CRE policy with Chainlink-based reference feeds and strict freshness/deviation guards.

38. Which path is easiest to misconfigure?

The easiest path to misconfigure is fixed-price mode.

39. Which path is most robust to latency, outages, and manipulation?

Most robust to latency/outage/manipulation is CRE consensus workflowing plus high-integrity onchain feed ingestion used as primary policy input.

40. Could stale but valid-looking prices cause systematically wrong bucket placement?

Yes. Stale but plausible prices can systematically place liquidity into wrong buckets.

41. Could oracle disagreement or feed/API drift create oscillation or bad rebalancing?

Yes. Source disagreement/drift can create oscillation and repeated bad reallocations.

42. Is there an attack where a stale/reference price causes the vault to move liquidity out of a safe bucket into a fragile one?

Yes. A stale reference can move capital from relatively safe buckets into fragile ranges.

G. Smart Contract Safety

43. Identify all privileged roles and what they can do.

Privileged roles are admin, keeper, and swapper with broad policy and operational influence.

44. Can admin, keeper, or swapper permissions directly or indirectly cause loss?

Yes. Admin/keeper/swapper permissions can directly or indirectly cause user loss via bad config, unsafe moves, or failed emergency handling.

45. Review:

- pause/unpause,
- cap changes,
- fee changes,
- min bucket index,
- move functions,
- buffer operations,
- any rescue/emergency paths.

Pause/unpause, cap/fee/min-bucket changes are admin-controlled; move and buffer operations are role-gated; recovery paths are operator-mediated and can keep vault paused.

46. Are there reentrancy, ordering, access control, stale-state, or partial-state-update risks?

Reentrancy guards exist; dominant residual risk is stale-state and partial economic correctness under asynchronous operation, not classic lock bypass.

47. Are there hidden assumptions in helper libraries or Buffer/VaultAuth interactions?

Yes. Helper-layer assumptions in buffer-ratio checks, conversions, and state freshness can fail under stressed timing and changing pool conditions.

48. Does any contract rely on optimistic assumptions about Ajna pool behavior that should be treated as unsafe?

Yes. Contract logic uses valuation proxies that may not equal immediate executable liquidity during Ajna stress states.

H. Adversarial Scenarios

49. Share price manipulation

Share price manipulation is plausible via timing around stale accounting and liquidity asymmetry; impact is usually fairness distortion unless stress persists (severity: Medium).

50. Donation / inflation attack

Donation/inflation attack surface is comparatively low in this design due to accounting mechanics, but invariants should remain tested (severity: Low).

51. Sandwiching around deposit/redeem

Sandwiching around deposit/redeem can worsen user execution fairness through keeper/state timing (severity: Medium).

52. Strategic buffer exhaustion

Strategic buffer exhaustion can force late-user redemption failures and strong first-exit advantage (severity: Critical).

53. Keeper timing exploitation

Keeper timing exploitation can induce repeated poor or skipped rebalances around predictable windows (severity: High).

54. Wrong-bucket reallocation via stale oracle/reference data

Stale oracle/reference data can drive wrong-bucket reallocations and realized yield loss (severity: High).

55. Griefing through small/dust bucket states

Dust-state griefing can create skip-heavy behavior and operational drag (severity: Medium).

56. Borrower-side toxic flow / adverse selection

Borrower-side toxic flow can extract value from passive liquidity placement (severity: High).

57. Auction / bankruptcy state trapping liquidity

Auction/bankruptcy states can trap liquidity and create severe liveness plus economic harm (severity: Critical).

58. Operator key compromise

Operator key compromise in bridge model can trigger unauthorized writes and broad operational damage (severity: High).

59. Misconfigured buffer ratio

Misconfigured buffer ratio can either break exits (too low) or severely reduce strategy efficacy (too high) (severity: High).

60. Misconfigured fixed price

Misconfigured fixed price can produce systematic bad policy decisions and wrong rebalances (severity: High).

61. CRE workflow bug causing repeated or skipped operational actions

CRE workflow bugs can repeat or skip actions, harming liveness and consistency if not caught by idempotency/invariants (severity: High).

62. AI advisory path influencing operations incorrectly

AI advisory misinfluence is a process risk; direct protocol risk is lower while AI remains non-authoritative (severity: Low).

63. Replay / duplication / double-execution risk in CRE-triggered workflows

Replay/duplication risk is partly mitigated by idempotency and dedupe, but residual race windows remain (severity: Medium).

64. Divergence between deterministic checks and advisory AI outputs

Deterministic vs AI divergence can cause human mis-triage unless runbooks explicitly prioritize deterministic signals (severity: Informational).

I. CRE-Specific Design Review

65. Can CRE safely replace the existing keeper loop, or should it only supervise / gate / queue keeper actions?

CRE should not fully replace keeper logic immediately; near-term safest model is CRE supervision/policy + constrained deterministic execution.

66. Which keeper responsibilities are good candidates for CRE?

Good CRE candidates: scheduling, monitoring, deterministic policy checks, queueing, checkpointing, and alerting.

67. Which responsibilities should remain deterministic and minimal?

Responsibilities that must stay deterministic/minimal include auth checks, action construction, idempotency, allowlists, and owner verification.

68. Which responsibilities should never rely on AI?

Never rely on AI for write authorization, safety gating, liquidation-sensitive actions, or emergency mutation decisions.

69. How should CRE be used:

- as a scheduler,
- as a monitor,
- as a policy engine,
- as a transaction orchestrator,

- as a human-in-the-loop escalation layer?

Use CRE as scheduler + monitor + deterministic policy engine + constrained transaction orchestrator + human escalation layer.

70. Review the Converge workflows and determine:

- what is already relevant to keeper supervision,
- what is reusable,
- what is only hackathon/demo quality,
- what would need hardening for production.

In current Converge workflows, runtime/queue/idempotency pieces are reusable; bridge-heavy and prototype native-write paths need production hardening.

71. Evaluate whether CRE adds:

- better determinism,
- better observability,
- better replay protection,
- better state checkpointing,
- or just more complexity.

CRE adds meaningful determinism, observability, replay protection, and checkpointing, but also introduces additional operational complexity.

72. Evaluate CRE-specific risks:

- workflow misconfiguration,
- secrets handling,
- trigger duplication,
- external HTTP dependency,
- runtime persistence mismatch,
- unsafe onchain write authority,
- deployment model risk,
- early-access platform risk.

Key CRE risks: workflow misconfig, secret handling, trigger duplication, HTTP bridge dependency, persistence mismatch, unsafe authority delegation, and platform maturity concerns.

73. Distinguish sharply between:

- simulation confidence,
- production readiness,
- and institutional-grade operational resilience.

Simulation provides strong pre-deploy confidence but is not identical to DON production behavior; institutional resilience still depends on hardened operations.

J. Stress Tests

74. 20%, 35%, and 50% collateral price shocks

At 20/35/50% collateral shocks, boundary regimes shift quickly; liveness pressure rises and higher-shock scenarios can realize impairment (severity: High/Critical).

75. rapid borrower deleveraging

Rapid deleveraging makes target buckets stale quickly and increases fairness/yield degradation risk (severity: High).

76. active liquidation auctions

Active liquidation auctions increase abort/refill delay probability and can degrade exit liveness (severity: High).

77. bad debt emergence

Bad debt emergence can lock system into prolonged no-rebalance states with severe liveness/economic impact (severity: Critical).

78. 25%, 50%, and 80% of vault TVL trying to redeem before next rebalance

If 25/50/80% TVL tries to redeem pre-rebalance, buffer exhaustion creates revert windows and first-exit advantage (severity: High/Critical).

79. stale oracle / bad price selection

Stale oracle or bad price selection can repeatedly misallocate liquidity and realize yield loss (severity: High).

80. keeper offline for 12h / 24h / 72h

Keeper offline for 12h/24h/72h progressively increases liveness and economic risk, with 72h potentially severe (severity: Medium/High/Critical).

81. subgraph failure

Subgraph failure can remove bad-debt visibility; fail-open configurations materially increase risk (severity: Medium/High).

82. admin mistake

Admin mistakes in config or role control can create immediate broad safety failures (severity: High).

83. CRE workflow outage

CRE workflow outage mainly harms liveness through missed windows and queue backlog (severity: Medium).

84. CRE workflow duplicated trigger

CRE duplicated trigger usually gets deduped, but residual races/retry churn remain possible (severity: Medium).

85. AI advisory endpoint returning nonsense while deterministic checks still pass

If AI advisory returns nonsense while deterministic checks still pass, users mostly see misleading alerts and recommendations rather than unsafe automated state changes.

- What users observe: Alert noise, contradictory guidance, and lower confidence in incident messaging.
- What breaks first: Operator triage quality (signal-to-noise), not deterministic execution safety.
- Loss type: Primarily liveness/operations degradation; usually not direct realized protocol loss while AI is advisory-only.
- Who bears cost: Operators first (response overhead), then users indirectly via slower/manual response quality.
- Recovery path: Operator-dependent but practical - suppress/disable AI advisory output and continue deterministic guardrails (severity: Low).